

JAVASCRIPT

LAB-1:

1.Lab Question: Alert, Confirm, and Prompt – Quiz Game

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Quiz Game</title>

</head>

<body>

  <script>

    alert("Welcome to the Quiz Game!");

    let userName = prompt("What's your name?");

    if (!userName) {

      userName = "Player"; }

    let isReady = confirm(userName + ", are you ready for the quiz?");

    if (isReady) {

      let answer = prompt(

        "Which planet is known as the Red Planet?\n" +

        "A: Earth\n" +

        "B: Mars\n" +

        "C: Jupiter\n" +

        "D: Venus");

      if (answer && answer.toLowerCase() === "b") {

        alert("🎉 Correct! Well done, " + userName + "!");

      } else { alert("❌ Oops! The correct answer is B: Mars.");}

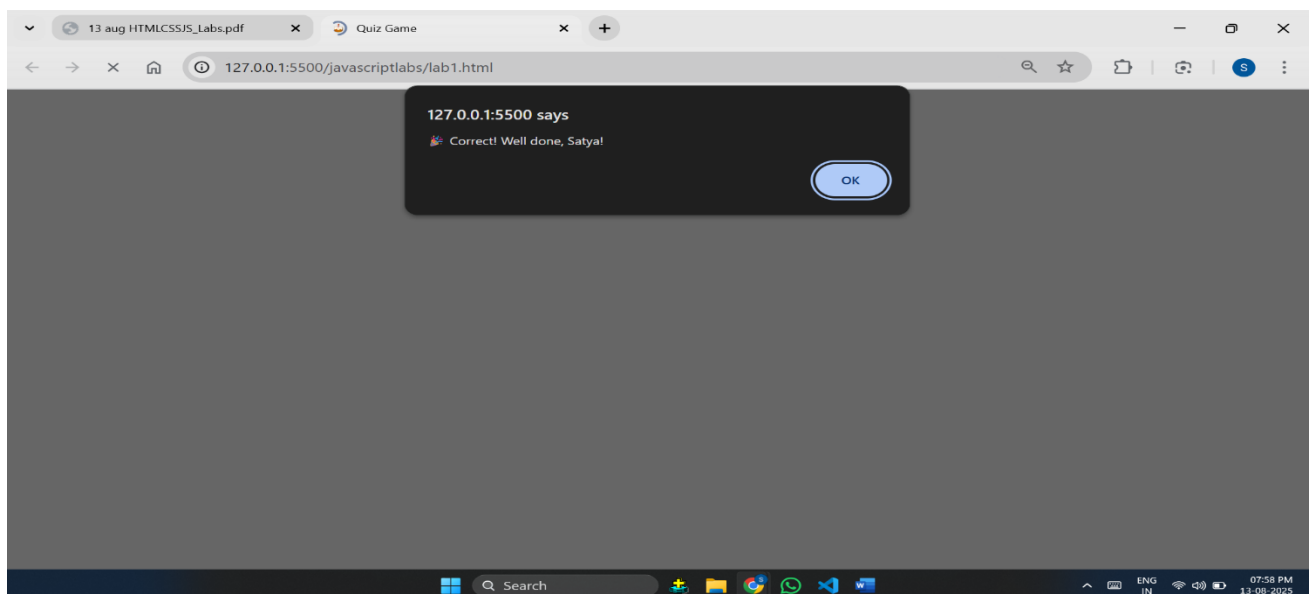
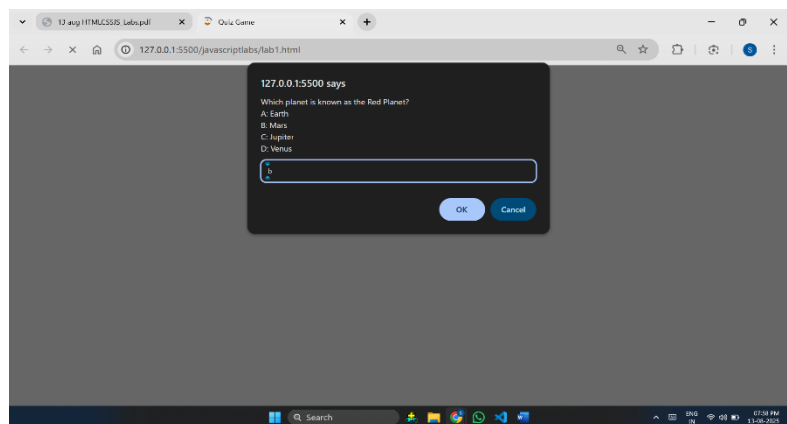
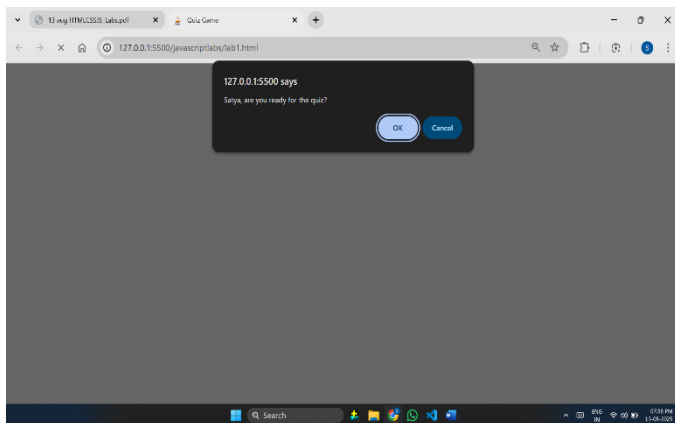
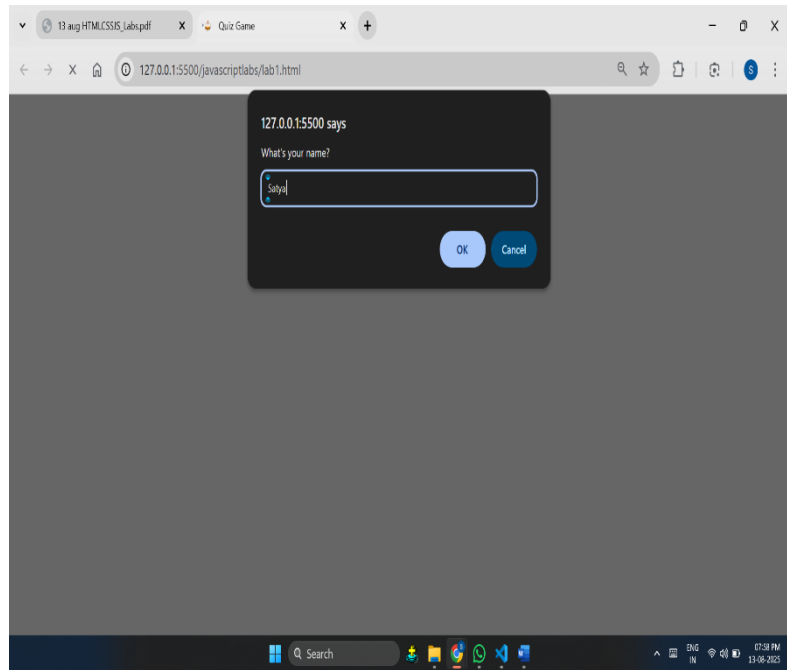
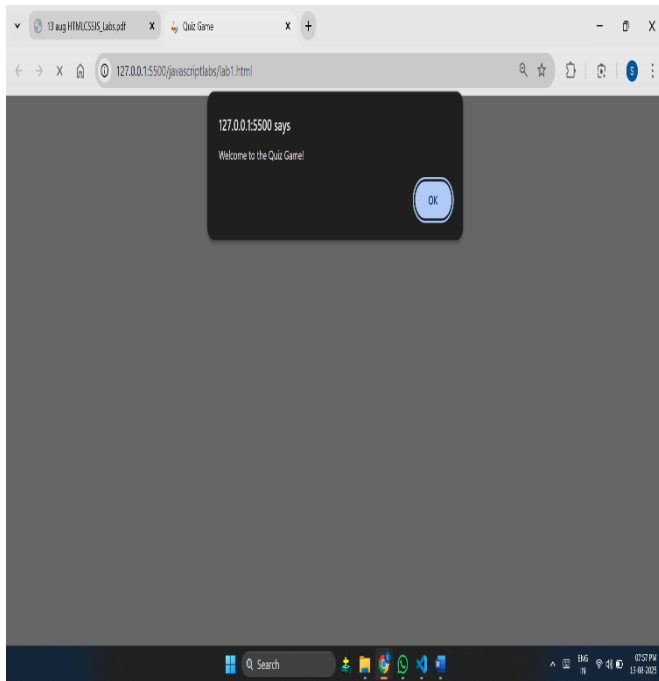
      else { alert("Okay " + userName + ", maybe next time!");}

    }

  </script>

</body></html>
```

OUTPUT:



LAB-2:

2.Lab Question: Income Tax Calculator Question: Write a JavaScript program to calculate an individual's income tax based on the following rules:

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Income Tax Calculator</title>

</head>

<body>

<script>

  let fullName = prompt("Enter your full name:").trim();

  let panNumber = prompt("Enter your PAN number (10 characters):").trim().toUpperCase();

  if (panNumber.length !== 10) {

    alert("❌ Invalid PAN Number! Must be exactly 10 characters.");

    throw new Error("Invalid PAN Number");

  }

  let dobInput = prompt("Enter your Date of Birth (YYYY-MM-DD):");

  let dob = new Date(dobInput);

  if (isNaN(dob.getTime())) {

    alert("❌ Invalid Date of Birth format!");

    throw new Error("Invalid Date");

  }

  let annualIncome = parseFloat(prompt("Enter your Annual Income in ₹:"));

  if (isNaN(annualIncome) || annualIncome < 0) {

    alert("❌ Invalid Income!");

    throw new Error("Invalid Income");

  }

}
```

```
let today = new Date();

let age = today.getFullYear() - dob.getFullYear();

let monthDiff = today.getMonth() - dob.getMonth();

if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < dob.getDate())) {

    age--;

}

let tax = 0;

if (annualIncome <= 250000) {

    tax = 0;

} else if (annualIncome <= 500000) {

    tax = (annualIncome - 250000) * 0.05;

} else if (annualIncome <= 1000000) {

    tax = (250000 * 0.05) + (annualIncome - 500000) * 0.20;

} else {

    tax = (250000 * 0.05) + (500000 * 0.20) + (annualIncome - 1000000) * 0.30;

}

tax = Math.round(tax);

let output =

    "Name: " + fullName.toUpperCase() + "\n" +

    "PAN: " + panNumber + "\n" +

    "Age: " + age + " years\n" +

    "Annual Income: ₹ " + annualIncome.toLocaleString() + "\n" +

    "Calculated Tax: ₹ " + tax.toLocaleString();

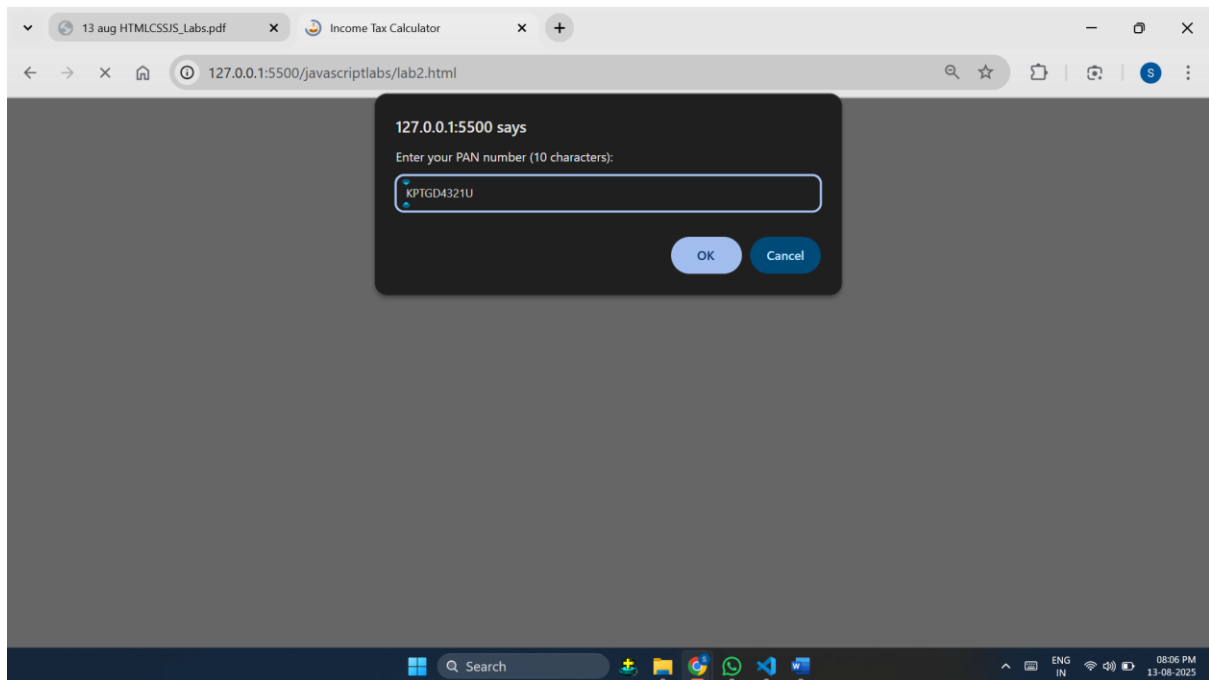
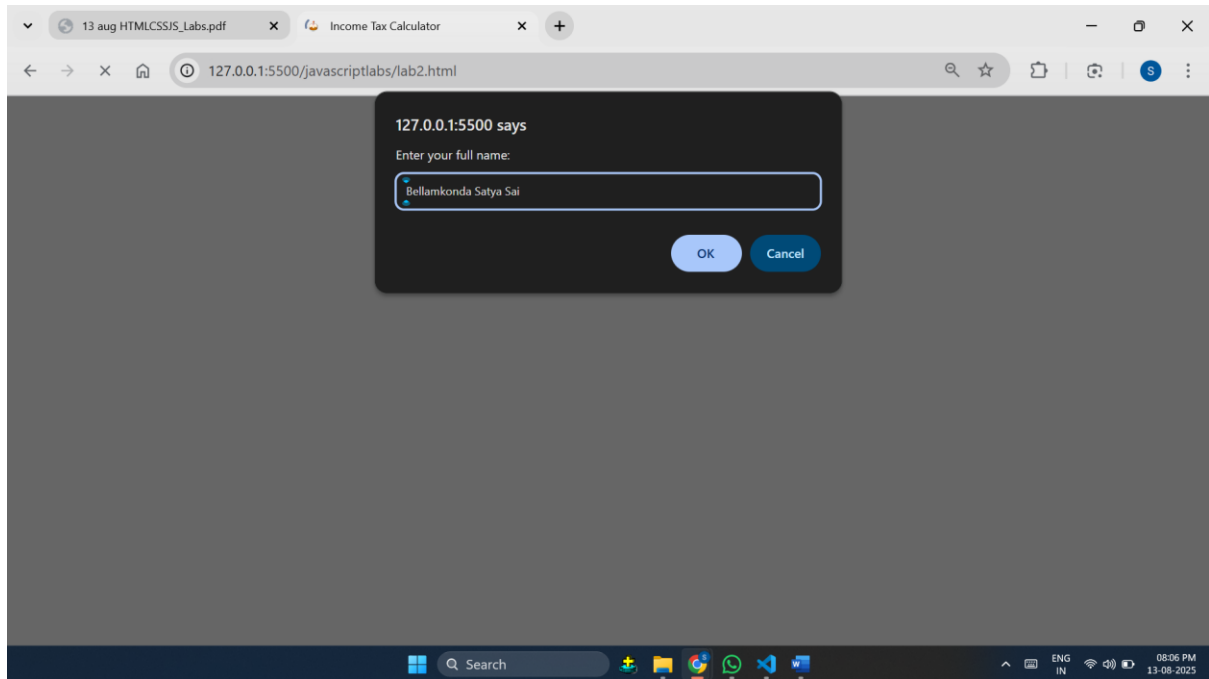
alert(output);

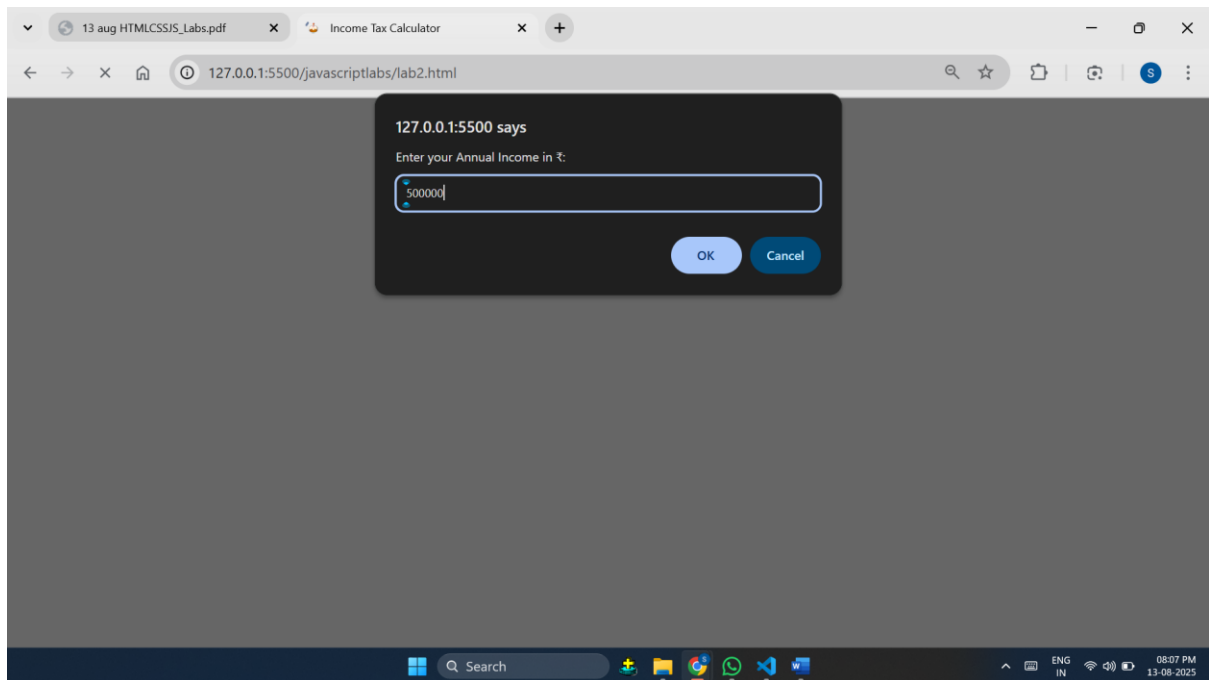
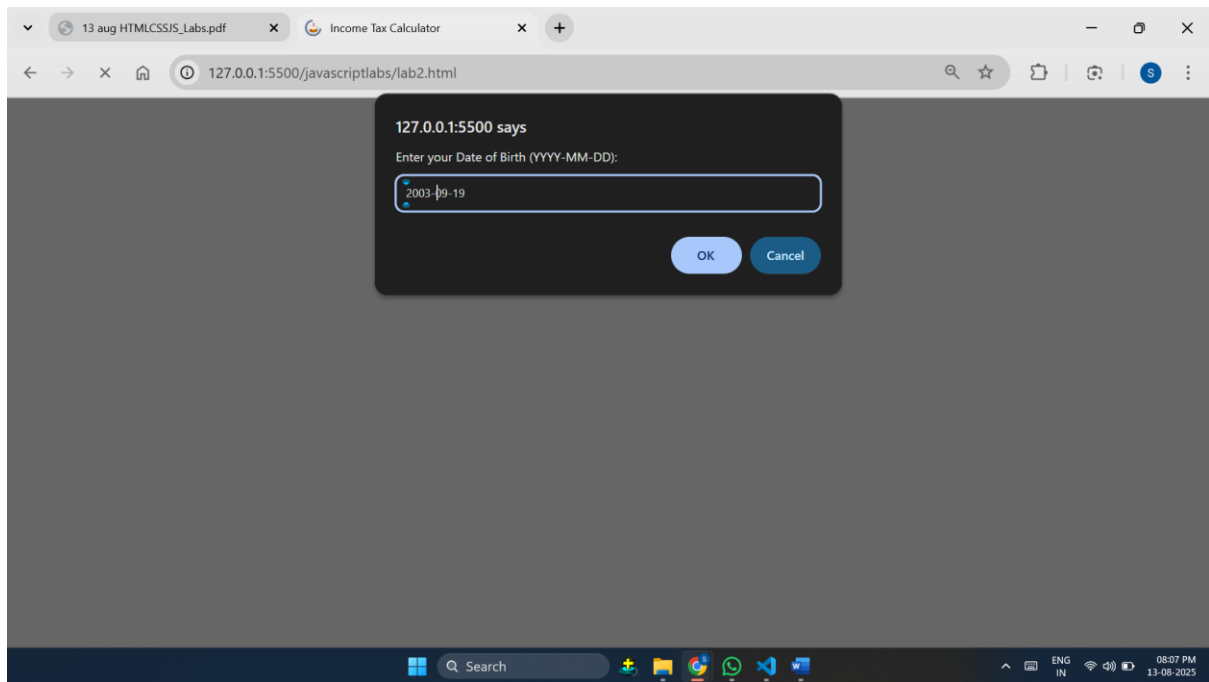
</script>

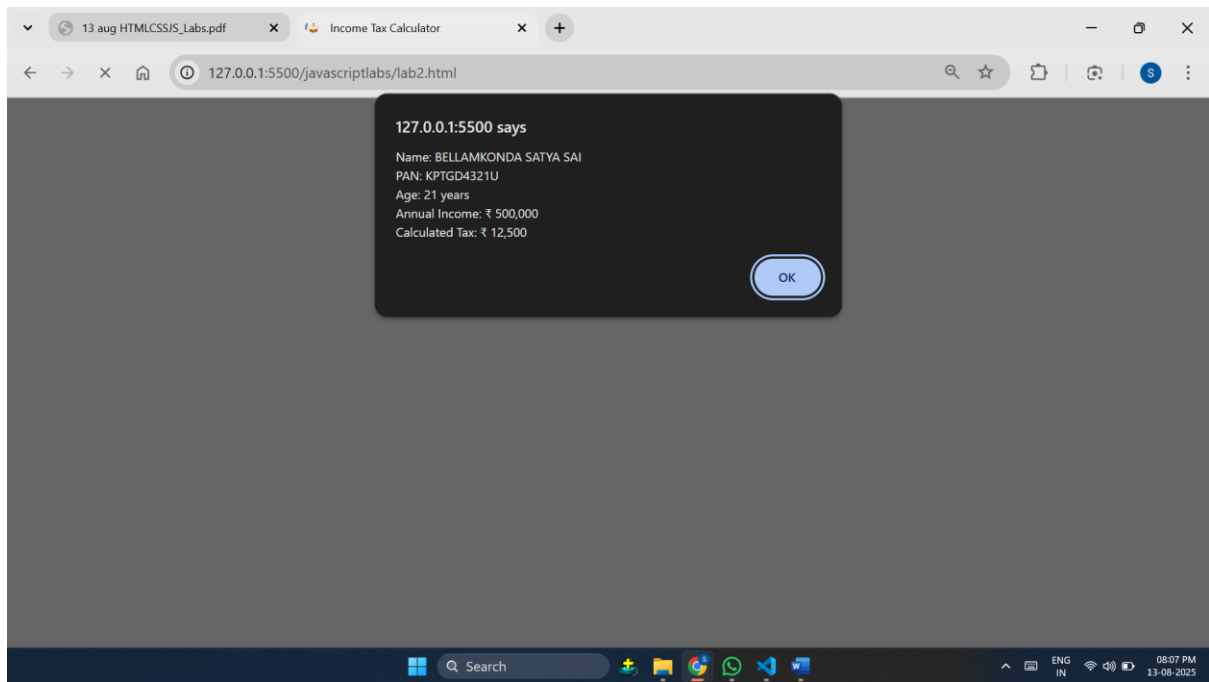
</body>

</html>
```

OUTPUT







LAB-3:

3.Lab Question: Equality & Inequality Checker

Question: Write a JavaScript program that asks the user to enter two values and then compares them using.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Equality Comparisons</title>
</head>
<body>
<script>
  let value1 = prompt("Enter the first value:");
```

```

let value2 = prompt("Enter the second value:");

let numValue1 = Number(value1);

let numValue2 = Number(value2);

let type1 = typeof value1;

let type2 = typeof numValue2;

let eq = (value1 == numValue2);

let strictEq = (value1 === numValue2);

let neq = (value1 != numValue2);

let strictNeq = (value1 !== numValue2);

let output = "";

output += `First Value: ${value1} (${type1})\n`;

output += `Second Value: ${value2} (string from prompt, then converted to number: ${numValue2} - type: ${typeof numValue2})\n\n`;

output += `Using == : ${eq} → Equality ignores type\n`;

output += `Using === : ${strictEq} → Strict equality checks type too\n`;

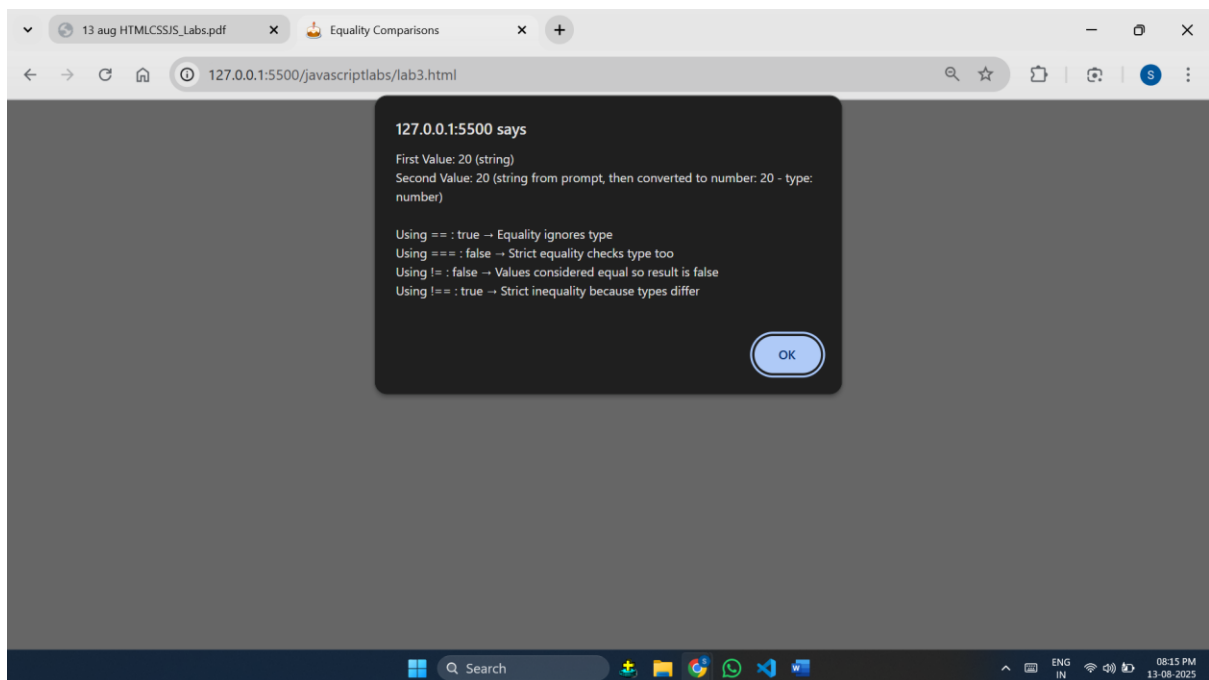
output += `Using != : ${neq} → Values considered equal so result is false\n`;

output += `Using !== : ${strictNeq} → Strict inequality because types differ\n`;

alert(output);
</script></body></html>

```

OUTPUT:



LAB-4:

4.Lab Question: Bank Customers – Dynamic Array
Creation & Manipulation Question: Write a
JavaScript program for a bank to store and manage
customer names.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Bank Customers</title>

</head>

<body>

<script>

  let numCustomers = parseInt(prompt("Enter the number of customers to register:"));

  if (isNaN(numCustomers) || numCustomers <= 0) {

    alert("Invalid number of customers!");

    throw new Error("Invalid input");

  }

  let customers = [];

  for (let i = 0; i < numCustomers; i++) {

    let name = prompt(`Enter name of customer ${i + 1}:`).trim();

    customers.push(name);

  }

  console.log("Original Customers:", customers.join(", "));

  console.log("Customers using for...of:");

  for (let cust of customers) {

    console.log(cust);

  }

  console.log("Customer IDs using for...in:");

  for (let id in customers) {
```

```

    console.log(id);
}

customers.push("Anita");

console.log("After push('Anita'):", customers.join(", "));

customers.pop();

console.log("After pop():", customers.join(", "));

customers.splice(1, 0, "Vikram"); // insert at index 1

console.log("After insert 'Vikram' at position 1:", customers.join(", "));

customers.splice(2, 1); // remove 1 item at index 2

console.log("After delete at position 2:", customers.join(", "));

console.log("Final Customers:", customers.join(", "));

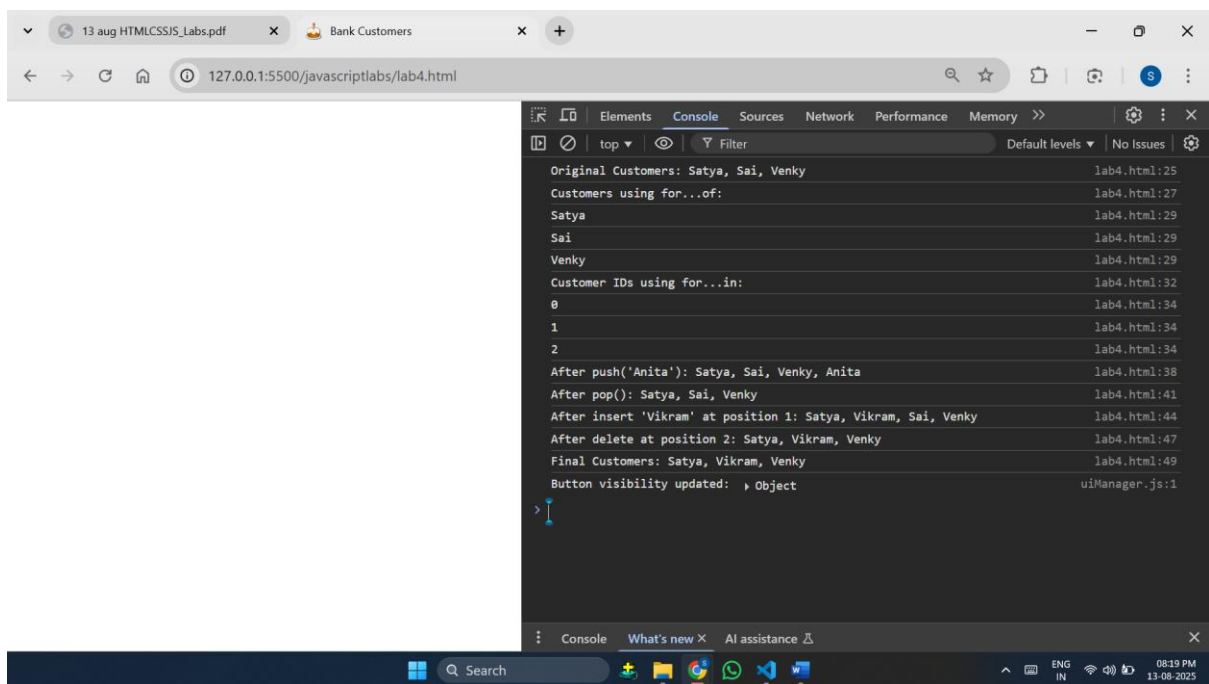
</script>

</body>

</html>

```

OUTPUT:



LAB-5:

Lab Question: JSON – Book Store Management

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Book Store Management</title>

</head>

<body>

<script>

  let books = [

    { title: "JavaScript Basics", author: "Anu", price: 450, available: true },

    { title: "HTML & CSS", author: "Devi", price: 350, available: false }

  ];

  console.log("All Books:");

  for (let book of books) {

    console.log(` Title: ${book.title}, Author: ${book.author}, Price: ${book.price} ₹, Available: ${book.available}` );

  }

  console.log("Available Books:");

  for (let book of books) {

    if (book.available) {

      console.log(` Title: ${book.title}, Author: ${book.author}, Price: ${book.price} ₹ `);

    }

  }

  books.push({ title: "Node.js Guide", author: "Ravi", price: 600, available: true });

  console.log("After Adding:");

  console.log(books.map(book => book.title).join(", "));

  let bookToUpdate = books.find(b => b.title === "JavaScript Basics");

  if (bookToUpdate) {

    bookToUpdate.price = 500;

  }

}
```

```

console.log("After Price Update:");

console.log(` ${bookToUpdate.title} - ${bookToUpdate.price} ₹`);

books = books.filter(b => b.title !== "JavaScript Basics");

console.log("After Deletion:");

console.log(books.map(book => book.title).join(", "));

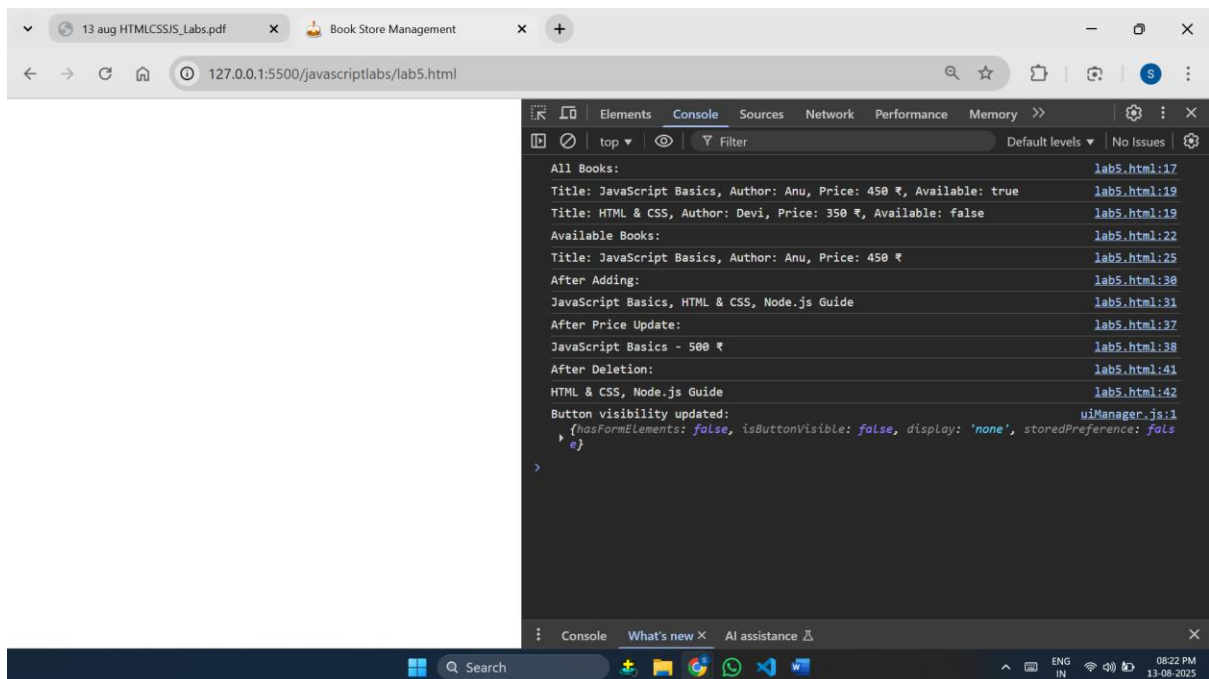
</script>

</body>

</html>

```

OUTPUT:



LAB-6:

6. Lab Question: Tour Booking Form – getElementById()

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Tour Booking Form</title>

</head>

<body>

  <h2>Tour Booking Form</h2>

  <form onsubmit="calculateBooking(event)">

    <label for="name">Full Name:</label>

    <input type="text" id="name" required><br><br>

    <label for="email">Email:</label>

    <input type="email" id="email" required><br><br>

    <label for="destination">Destination:</label>

    <select id="destination" required>

      <option value="Ooty">Ooty</option>

      <option value="Goa">Goa</option>

      <option value="Manali">Manali</option>

    </select><br><br>

    <label for="days">Number of Days:</label>

    <input type="number" id="days" min="1" required><br><br>

    <label for="travelers">Number of Travelers:</label>

    <input type="number" id="travelers" min="1" required><br><br>

    <button type="submit">Book Now</button>

  </form>

  <h3>Booking Summary:</h3>

  <div id="summary"></div>
```

```
<script>

function calculateBooking(event) {

    event.preventDefault();

    let name = document.getElementById("name").value;

    let email = document.getElementById("email").value;

    let destination = document.getElementById("destination").value;

    let days = parseInt(document.getElementById("days").value);

    let travelers = parseInt(document.getElementById("travelers").value);

    let pricePerDay;

    if (destination === "Ooty") {

        pricePerDay = 2000;

    } else if (destination === "Goa") {

        pricePerDay = 3000;

    } else if (destination === "Manali") {

        pricePerDay = 4000;

    }

    let totalPrice = pricePerDay * days * travelers;

    document.getElementById("summary").innerHTML = `

        Name: ${name} <br>

        Email: ${email} <br>

        Destination: ${destination} <br>

        Days: ${days} <br>

        Travelers: ${travelers} <br>

        Total Price: ₹ ${totalPrice.toLocaleString()}

    `;

}

</script>

</body>

</html>
```

OUTPUT:

The screenshot shows a web browser window with the title 'Tour Booking Form'. The address bar displays '127.0.0.1:5500/javascriptlabs/lab6.html'. The form contains the following fields and values:

- Full Name: Bellamkonda Satya Sai Ven
- Email: satya@gmail.com
- Destination: Manali (selected from a dropdown menu)
- Number of Days: 10
- Number of Travelers: 2

A 'Book Now' button is located below the form fields.

Booking Summary:

This screenshot shows the same 'Tour Booking Form' as above, but with the 'Booking Summary' section expanded. The summary displays the following information:

- Name: Bellamkonda Satya Sai
- Email: satya@gmail.com
- Destination: Manali
- Days: 10
- Travelers: 2
- Total Price: ₹ 80,000

LAB-7:

Lab Question – Sports Team Player Details

Tasks: 1. Use object destructuring to extract name, sport, and the nested properties matches, runs, and average from player. 2. Using template literals, print the message in this exact format: Player Virat Kohli plays Cricket. Matches: 500 | Runs: 25000 | Batting Average: 57.2 3. Add another player object to an array and use array destructuring to print details for both players in a similar format.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Sports Team Player Details</title>

</head>

<body>

<script>

  const player = {

    name: "Virat Kohli",

    age: 36,

    sport: "Cricket",

    stats: {

      matches: 500,

      runs: 25000,

      average: 57.2

    }

  };


```

```

const { name, sport, stats: { matches, runs, average } } = player;

const player2 = {
  name: "Ronaldo",
  age: 37,
  sport: "Football",
  stats: {
    matches: 800,
    goals: 720,
    assists: 300 } };

const players = [player, player2];

const [p1, p2] = players;

console.log(` Player ${p1.name} plays ${p1.sport}. `);

console.log(` Matches: ${p1.stats.matches} | Runs: ${p1.stats.runs} | Batting Average: ${p1.stats.average} `);

console.log(` Player ${p2.name} plays ${p2.sport}. `);

console.log(` Matches: ${p2.stats.matches} | Goals: ${p2.stats.goals} | Assists: ${p2.stats.assists} `);

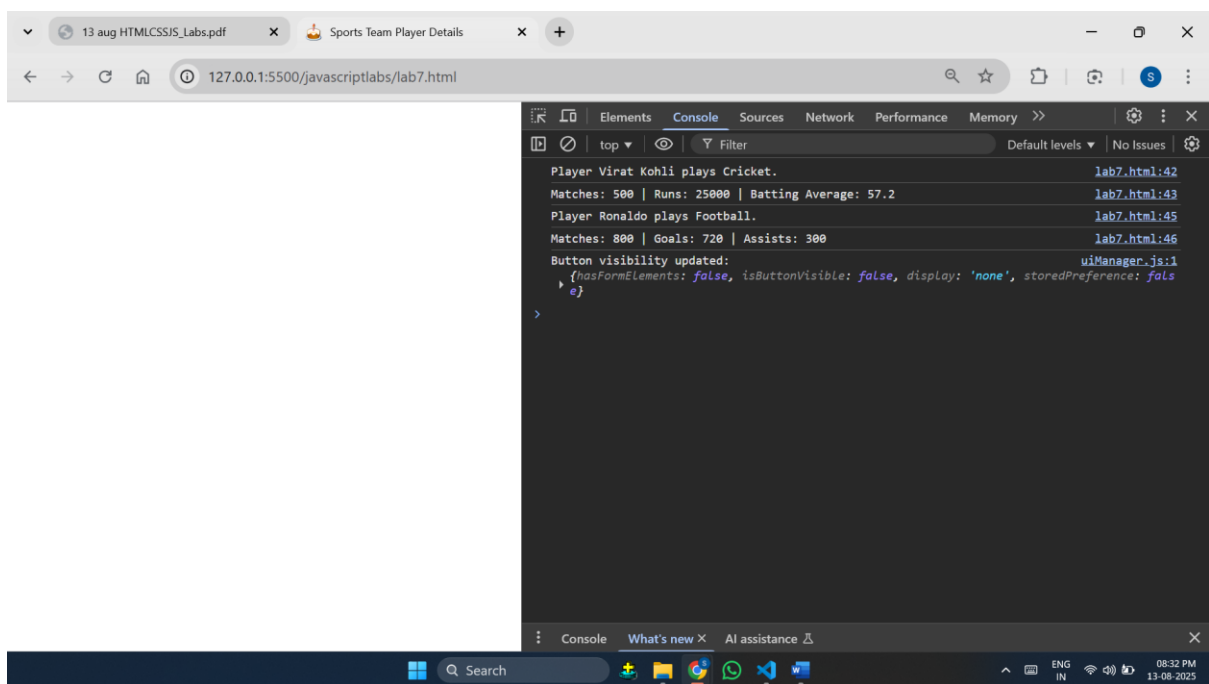
</script>

</body>

</html>

```

OUTPUT:



LAB-8:

Lab Question – Switchboard Control (let, var, const)
A building has a switchboard to control lights, fans, and AC. You need to write a JavaScript program to simulate turning them ON/OFF using let, var, and const.

CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Switchboard Control</title>

</head>

<body>

  <h1>Switchboard Control</h1>

  <script>

    const buildingName = "Sunrise Apartments";

    let deviceStatus = "OFF";

    var counter;

    const devices = ["Light", "Fan", "AC"];

    for (counter = 0; counter < devices.length; counter++) {

      deviceStatus = (deviceStatus === "OFF") ? "ON" : "OFF";

      console.log(` Checking Switch ${counter + 1}: ${devices[counter]} is ${deviceStatus}`);

    }

    console.log(` All switches checked for ${buildingName}.`);

    console.log(` Outside loop: var counter is ${counter}`);

    try {

      buildingName = "Moonlight Apartments";

    } catch (err) {

      console.error("Error:", err.message);

    }

  }

</script>

</body>

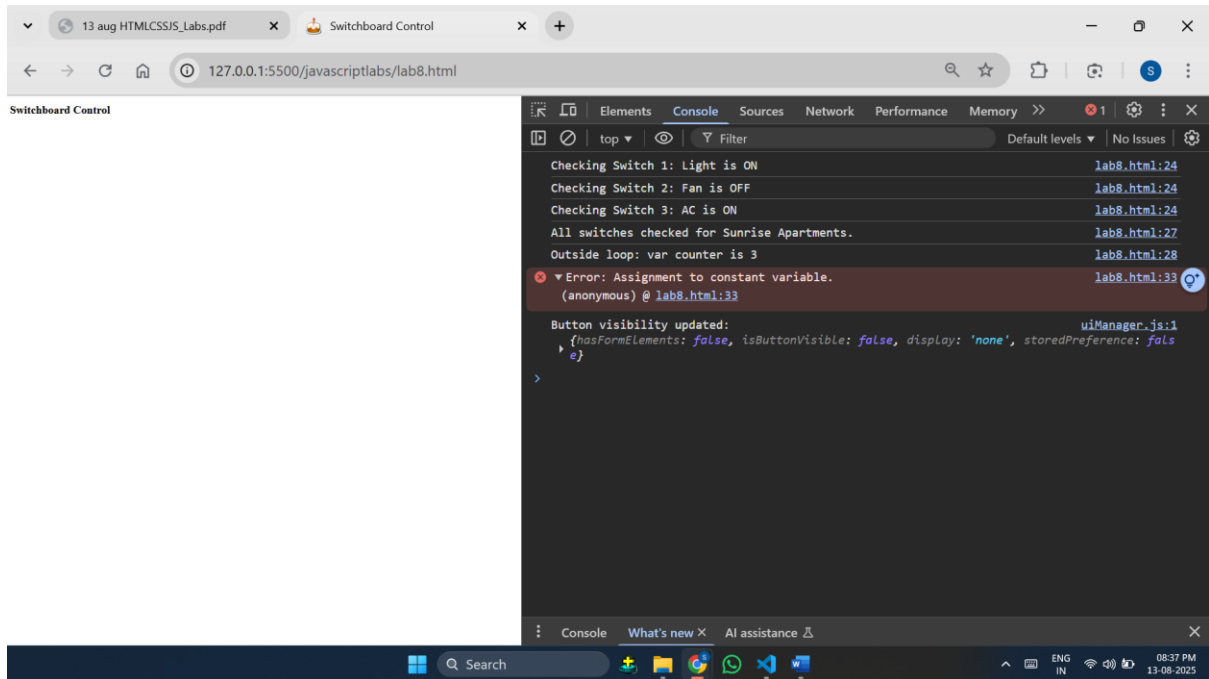
</html>
```

</script>

</body>

</html>

OUTPUT:



LAB-9:

9.Lab Question: Income Tax Calculator with Multiple Callbacks Question:

CODE:

```
<!DOCTYPE html>

<html>

<head>

  <title>Income Tax Calculator (Multiple Callbacks)</title>

</head>

<body>

  <h1>Income Tax Calculator</h1>

  <label>Name:</label>

  <input type="text" id="name"><br><br>

  <label>PAN:</label>

  <input type="text" id="pan"><br><br>

  <label>Annual Income:</label>

  <input type="number" id="income"><br><br>

  <button onclick="startCalculation()">Calculate Tax</button>

  <h2>Result:</h2>

  <div id="result"></div>

  <script>

    let taxRecords = [];

    function calculateTax(name, pan, income, callback) {

      let tax = 0;      if (income <= 250000) {

        tax = 0;

      } else if (income <= 500000) {

        tax = (income - 250000) * 0.05;

      } else if (income <= 1000000) {

        tax = (250000 * 0.05) + (income - 500000) * 0.2;

      } else {

        tax = (250000 * 0.05) + (500000 * 0.2) + (income - 1000000) * 0.3;

      }

    }
```

```

        callback({ name, pan, income, tax });
    }

    function logToConsole(details) {
        console.log(` Name: ${details.name}`);
        console.log(` PAN: ${details.pan}`);
        console.log(` Annual Income: ₹ ${details.income}`);
        console.log(` Calculated Tax: ₹ ${details.tax}`);
    }

    function displayOnPage(details) {
        document.getElementById("result").innerHTML = `
            <strong>Name:</strong> ${details.name} <br>
            <strong>PAN:</strong> ${details.pan} <br>
            <strong>Annual Income:</strong> ₹ ${details.income} <br>
            <strong>Calculated Tax:</strong> ₹ ${details.tax}
        `;
    }

    function saveToRecords(details) {
        taxRecords.push(details);
        console.log("Tax Records:", taxRecords);
    }

    function startCalculation() {
        let name = document.getElementById("name").value;
        let pan = document.getElementById("pan").value;
        let income = parseFloat(document.getElementById("income").value);

        calculateTax(name, pan, income, function(details) {
            logToConsole(details);
            displayOnPage(details);
            saveToRecords(details);
        });
    }
}

</script>
</body>
</html>

```

OUTPUT:

The screenshot shows a web browser window with the URL `127.0.0.1:5500/javascriptlabs/lab9.html`. The page title is "Income Tax Calculator". The form contains the following fields:

- Name:
- PAN:
- Annual Income:
-

The "Result:" section is empty. The browser's developer console is open, showing the following log entries:

```
Button visibility updated: uiManager.js:1
  {hasFormElements: true, isButtonVisible: false, display: 'none', storedPreference: false}
Button visibility updated: uiManager.js:1
  {hasFormElements: true, isButtonVisible: false, display: 'none', storedPreference: false}
```

The screenshot shows the same web browser window, but now the "Result:" section displays the calculated values:

Result:

Name: Bellamkonda Satya Sai
PAN: PLKPK98745R
Annual Income: ₹ 500000
Calculated Tax: ₹ 12500

The browser's developer console is open, showing the following log entries:

```
Button visibility updated: uiManager.js:1
  {hasFormElements: true, isButtonVisible: false, display: 'none', storedPreference: false}
Button visibility updated: uiManager.js:1
  {hasFormElements: true, isButtonVisible: false, display: 'none', storedPreference: false}
Name: Bellamkonda Satya Sai lab9.html:46
PAN: PLKPK98745R lab9.html:47
Annual Income: ₹ 500000 lab9.html:48
Calculated Tax: ₹ 12500 lab9.html:49
Tax Records: {[]} lab9.html:63
```

LAB-10:

Build a multi-section RTO Vehicle Registration form with Bootstrap that validates:

OUTPUT:

The screenshot shows a web browser window with the title "RTO Vehicle Registration Form". The URL is "127.0.0.1:5500/javascriptlabs/lab10.html". The form is titled "RTO Vehicle Registration Form" and contains a section titled "1. Owner Details". This section has the following fields:

- Full Name: Bellamkonda Satya Sai
- Email: satya@gmail.com
- Mobile (India): 9874563214
- Date of Birth: 19-09-2003
- Aadhaar: 987456321456
- PAN: HTGYR84521P

Below the "1. Owner Details" section are seven more sections, each with a dropdown arrow:

2. Address
3. Vehicle Details
4. Compliance & Insurance
5. Finance (Hypothecation)
6. Uploads
7. Declaration

The screenshot shows the same web browser window, but the form is now titled "RTO Vehicle Registration" and includes a message: "Please fill out all sections below to complete your application." The "1. Owner Details" section is expanded, and each field now has a green checkmark indicating successful validation:

- Full Name: Bellamkonda Satya Sai ✓
- Email: satya@gmail.co ✓
- Mobile Number: 9874564512 ✓
- Date of Birth: 19-09-2003 ✓
- Aadhaar Number: 987456321456 ✓
- PAN Number: PLKUH5432P ✓

Below the "1. Owner Details" section are seven more sections, each with a dropdown arrow:

2. Address Details
3. Vehicle Details
4. Compliance Details
5. Finance Details (Optional)
6. Upload Documents
7. Declarations

At the bottom of the form is a blue button labeled "Submit Application".

127.0.0.1:5500/javascriptlabs/lab10.html

RTO Vehicle Registration

Please fill out all sections below to complete your application.

100%

1. Owner Details

2. Address Details

3. Vehicle Details

4. Compliance Details

5. Finance Details (Optional)

6. Upload Documents

7. Declarations

Address Line 1

Kollipara

Address Line 2 (Optional)

Kollipara

City

Kollipara

State

Andhra Pradesh

PIN

845612

Submit Application

127.0.0.1:5500/javascriptlabs/lab10.html

RTO Vehicle Registration

Please fill out all sections below to complete your application.

100%

1. Owner Details

2. Address Details

3. Vehicle Details

4. Compliance Details

5. Finance Details (Optional)

Registration No.

AP-89-PC-9546

Class

Light Motor Vehicle

Please provide the vehicle registration number.

Fuel Type

☒ Petrol ☐ Diesel ☐ Electric

Make/Model

Maruti Suzuki Swift

Chassis No.

UA3D1B4S856789123

Engine No.

P12B56789123

Year of Manufacture

2020

Purchase Date

13-05-2023

Color

Blue

4. Compliance Details

5. Finance Details (Optional)

13 aug HTMLCSSJS_Labs.pdf | Google Gemini | RTO Vehicle Registration

127.0.0.1:5500/javascriptlabs/lab10.html

RTO Vehicle Registration

Please fill out all sections below to complete your application.

67%

1. Owner Details

2. Address Details

3. Vehicle Details

4. Compliance Details

5. Finance Details (Optional)

6. Upload Documents

7. Declarations

Submit Application

4. Compliance Details

Emission Norm	Insurance Policy No.
BS-IV	K3SPOL-987654321
Insurer Name	Insurance Expiry Date
HDFC	15-06-2029
PUC Valid Till	
15-06-2026	

13 aug HTMLCSSJS_Labs.pdf | Google Gemini | RTO Vehicle Registration

127.0.0.1:5500/javascriptlabs/lab10.html

RTO Vehicle Registration

Please fill out all sections below to complete your application.

67%

1. Owner Details

2. Address Details

3. Vehicle Details

4. Compliance Details

5. Finance Details (Optional)

6. Upload Documents

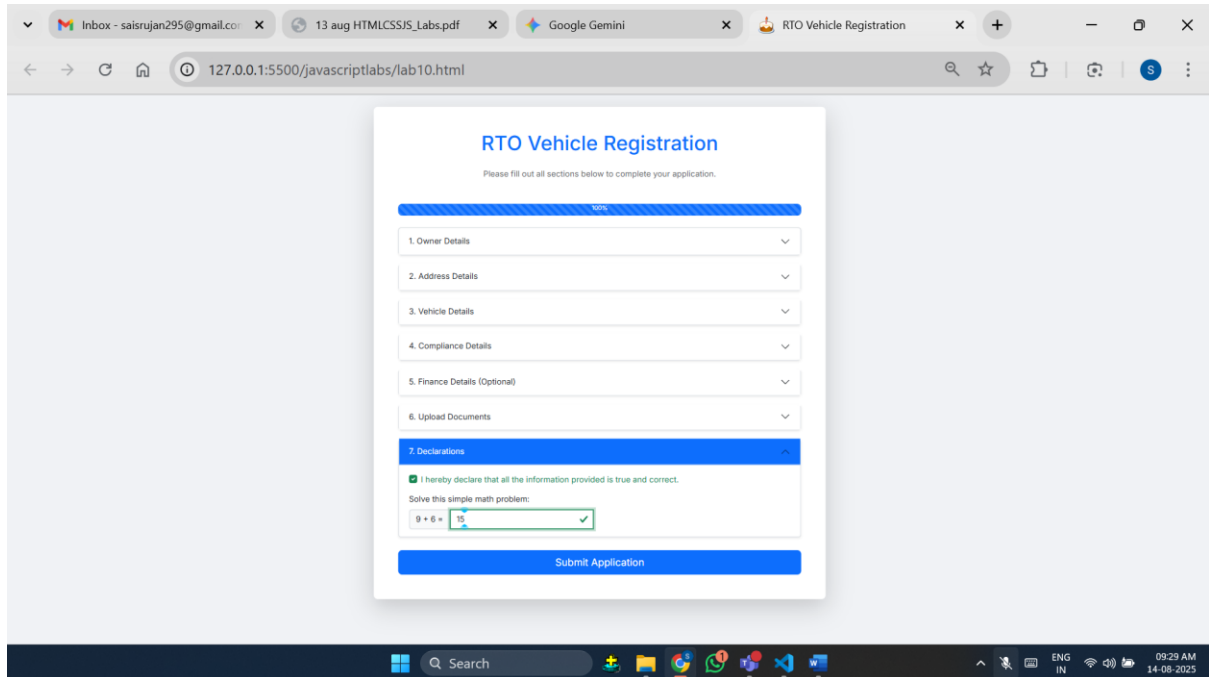
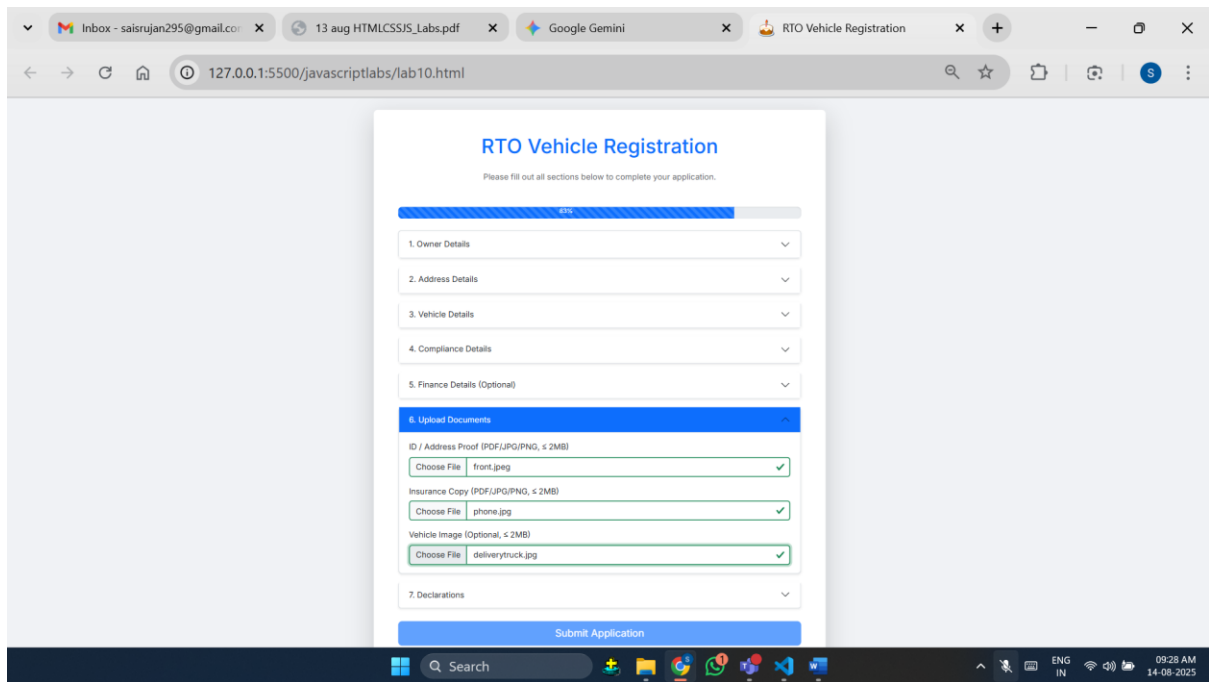
7. Declarations

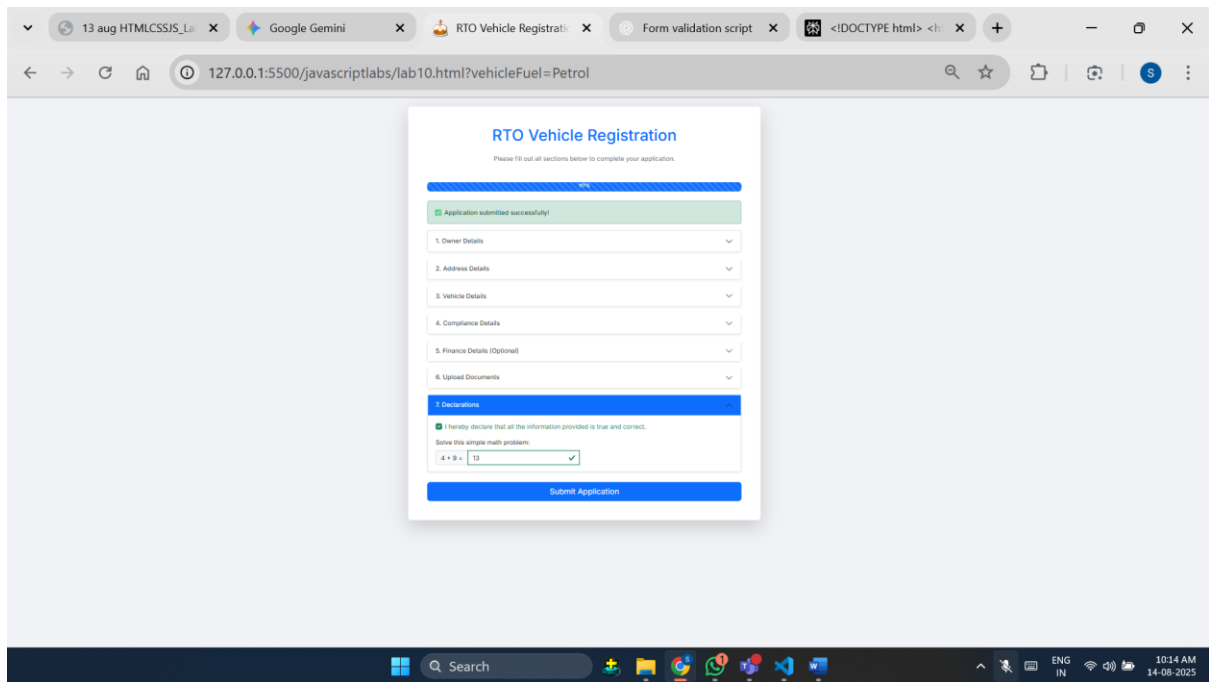
Submit Application

5. Finance Details (Optional)

☒ Hypothecation

Bank/NBFC Name	Agreement Date
HDFC	15-09-2027





LAB-11:

11. Lab Question – JavaScript Closure (Share Market Example) Objective: Demonstrate the concept of closures in JavaScript by implementing a simple Share Market Portfolio Tracker.

CODE:

```
<!DOCTYPE html>

<html>

<head>

  <title>Share Market Portfolio Tracker</title>

</head>

<body>

  <h2>JavaScript Closure - Share Market Portfolio Tracker</h2>

  <script>

    function createPortfolio() {

      let portfolio = [];
```

```

function buyShare(company, quantity, price) {
  portfolio.push({ company, quantity, price });
  console.log(` Bought ${quantity} shares of ${company} at ₹${price} each.`);
}

function sellShare(company, quantity) {
  let share = portfolio.find(s => s.company === company);
  if (share) {
    if (quantity <= share.quantity) {
      share.quantity -= quantity;
      console.log(` Sold ${quantity} shares of ${company}.`);
      if (share.quantity === 0) {
        portfolio = portfolio.filter(s => s.company !== company);
      }
    } else {
      console.log(` Not enough shares of ${company} to sell.`);
    }
  } else {
    console.log(` ${company} not found in portfolio.`);
  }
}

function totalValue() {
  let total = portfolio.reduce((sum, share) => sum + (share.quantity * share.price), 0);
  console.log(` Portfolio Value: ₹${total}`);
}

return {
  buyShare,
  sellShare,
  totalValue
};
}

const myPortfolio = createPortfolio();
myPortfolio.buyShare("TCS", 10, 3500);
myPortfolio.buyShare("Infosys", 5, 1500);
myPortfolio.sellShare("TCS", 3);
myPortfolio.totalValue();

```

```
</script>
</body>
</html>
```

OUTPUT:

